Magpie — Sprint 5 Technical Spec (Phase 3: Listing Draft Builder)

Audience: implementing agent (Codex) + Regan. **Builds on** Phases 1–2 (`SPRINT_0_1_SPEC.md`, `SPRINT_2_SPEC.md`, `SPRINT_3_SPEC.md`, `SPRINT_4_SPEC.md`). **Status: planning / review — implement only after this spec is reviewed and approved.**

> Build only what is in this document (the locked Sprint 5 scope, D-47…D-55). **Zero eBay calls of any kind** — no API, no OAuth, no metadata or required-specifics lookups, no publishing. The output of this sprint is a human-editable draft + export pack that Regan pastes/uploads into eBay's own web UI. Closing Sprint 5 closes Phase 3.

Conventions inherited (not restated): `TimeStampedUUIDModel` base; modular monolith under `apps/`; DRF defaults (pagination 50, filters, `SessionAuthentication` + `IsAuthenticated`); Vite dev proxy; `manage.py backup` must include all new tables; the schema/profile registry and `fields()` descriptors from Sprint 4; photo pipeline (processed/compressed variants) from Sprint 1.

---

0. Locked decisions this spec implements

- **D-47** Sprint 5 = Phase 3 in **one sprint**: `apps/listing` + `ListingDraft` + `ListingBoilerplate` + template generators + item-specifics builder + readiness checker (incl. leak scan) + clipboard/zip export + cockpit Listing panel. **No eBay calls.**
- **D-48** `ListingDraft` shape (multi-draft per item; `channel` + `channel_data` as the D-11 seam — **no `SalesChannel` model**; price provenance in `generated_meta`, **not an FK**; **no GST field** — price is the buyer-facing GST-inclusive number per ACCC total-price guidance).
- **D-49** `ListingBoilerplate`: DB-backed, admin-edited, per-channel (the `FeeSchedule` pattern).
- **D-50** Generators: deterministic per-profile templates; fail-soft; regenerate never silently overwrites human edits; fixed description structure; optional SKU footer

behind a toggle; **no AI / no HTTP** (F-09 + Phase 5).

- **D-51** Generated HTML uses a **strict safe-tag whitelist** — no anchors, scripts, active content, or external images (eBay AU links/active-content policy, verified at planning time).
- **D-52** Privacy/internal-data guard is **structural**: generators and the specifics builder cannot consume blocked fields (allowlisted context); leak scan = literal `imei` / `serial_no` values only (readiness **fail**); cost/profit protection is structural + test-asserted (no prose-scanning for dollar amounts). `cert_no` allowed; SKU only via the toggle.
- **D-53** Readiness checker: pure function → fails/warns table below; `ready` requires zero fails; **export allowed from any status** with an obvious unready badge; **"mark item listed" is manual with confirmation, never automatic.**
- **D-54** Export = clipboard blocks + zip bundle (`title.txt` , `subtitle.txt` if present, `description.html` , `specifics.csv` , `readiness.json` , `listing-summary.json` as the Phase 4 seam, ordered **processed** photos named `01_<sku>.jpg` …; missing photo IDs skipped + reported).
- **D-55** Out of scope (see §12).

---

1. New app & repo additions

```
backend/apps/listing/              # NEW
├── constants.py                   # EBAY_TITLE_MAX, ALLOWED_HTML_TAGS, BLOCKED_FIE
├── models.py                      # ListingDraft, ListingBoilerplate
├── context.py                     # safe_context(item) — the structural privacy gu
├── generators.py                  # per-profile title/description generators + reg
├── specifics.py                   # build_specifics(item)
├── readiness.py                   # check_readiness(draft) + leak scan
├── export.py                      # zip bundle builder
├── serializers.py
├── views.py
└── tests/
```

- `INSTALLED_APPS += "apps.listing"` . New migrations expected this sprint (two tables).
- **No new third-party dependencies. No HTTP client imports anywhere in `apps/listing` .**

- **Frontend: new `ListingPanel` section on the existing Item Detail screen + `listing.ts` API module. No new routes.**

Dependency direction: `listing` → `inventory` / `valuation` (read-only). Nothing imports `listing`.

---

2. Constants & models

2.1 `listing/constants.py`

```
EBAY_TITLE_MAX = 80   # characters — enforced at READINESS / ready-transition, no

# Tags generators may emit (D-51). NO <a>, <img>, <script>, <style>, <iframe>, fo
ALLOWED_HTML_TAGS = {"h2", "h3", "p", "ul", "ol", "li", "strong", "em",
                     "table", "tbody", "tr", "th", "td", "br"}

# Item fields / attribute keys that generated listing text may NEVER consume (D-5
# Enforcement is structural via safe_context() (§3.1); this set also drives the t
BLOCKED_FIELDS = {
    # device identifiers
    "imei", "serial_no",
    # internal economics
    "acquisition_cost", "purchase_price", "refurb_cost", "true_cost",
    "inbound_shipping_cost", "est_outbound_shipping", "est_packaging_cost",
    "min_price", "min_acceptable_price", "fast_sale_price", "patient_price",
    "estimated_value", "target_price",
    # internal operations / provenance
    "location", "storage_location", "acquisition_source", "supplier",
    "notes", "confidence_reason", "confidence_score", "override_reason",
}
# cert_no is explicitly ALLOWED (public verification reference — a selling point)
# SKU is allowed ONLY via the sku-footer toggle (D-50), never inside templates.
```

2.2 `listing.ListingDraft`

```
class ListingDraft(TimeStampedUUIDModel):
    class Status(models.TextChoices):
        DRAFT = "draft", "Draft"
        READY = "ready", "Ready"
        EXPORTED = "exported", "Exported"
        # Phase 4 adds published / publish_failed / revise_needed etc.
```

```
    class Format(models.TextChoices):
        FIXED = "fixed", "Fixed price"
        AUCTION = "auction", "Auction"

    item = models.ForeignKey("inventory.InventoryItem", on_delete=models.CASCADE,
                             related_name="listing_drafts")          # multiple d
    status = models.CharField(max_length=10, choices=Status.choices, default=Stat
    channel = models.CharField(max_length=30, default="ebay_au")     # D-11 seam
    channel_data = models.JSONField(default=dict, blank=True)

    title = models.CharField(max_length=200, blank=True, default="")  # soft cap;
    subtitle = models.CharField(max_length=200, blank=True, default="")
    description_html = models.TextField(blank=True, default="")
    listing_format = models.CharField(max_length=10, choices=Format.choices, defa

    price = models.DecimalField(max_digits=12, decimal_places=2, null=True, blank
    currency = models.CharField(max_length=3, default="AUD")
    quantity = models.PositiveIntegerField(default=1)
    est_shipping_note = models.CharField(max_length=200, blank=True, default="")

    item_specifics = models.JSONField(default=list, blank=True)   # [{ "name": st
    photo_ids = models.JSONField(default=list, blank=True)        # ordered ItemP
    include_sku_footer = models.BooleanField(default=False)       # D-50/D-52 SKU
    boilerplate = models.ForeignKey("listing.ListingBoilerplate", null=True, blan
                                    on_delete=models.SET_NULL, related_name="draf

    title_edited = models.BooleanField(default=False)             # human edit tr
    description_edited = models.BooleanField(default=False)
    generated_meta = models.JSONField(default=dict, blank=True)   # see §2.3
    exported_at = models.DateTimeField(null=True, blank=True)

    class Meta:
        ordering = ["-created_at"]
        indexes = [models.Index(fields=["item"]), models.Index(fields=["status"])
```

No GST field (D-48). `photo_ids` **is plain ordered JSONB — a working document, not an evidentiary relation; missing IDs are tolerated at render/export time and surfaced by readiness.**

2.3 `generated_meta` shape (light provenance — not FKs)

```
{
  "title":       {"template_key": "phones", "version": 1, "generated_at": "<iso>
  "description": {"template_key": "phones", "version": 1, "generated_at": "<iso>
  "price_source": {"valuation_report_id": "<uuid>", "price_point": "suggested",
```

```
                    "value": "120.00", "captured_at": "<iso>"}
}
```

`price_source` **is written when the draft's price is defaulted from the item's current** `ValuationReport` **(Q-10 caveat 1: metadata snapshot, deliberately not an FK).**

2.4 `listing.ListingBoilerplate` (admin-managed — the `FeeSchedule` pattern)

```
class ListingBoilerplate(TimeStampedUUIDModel):
    channel = models.CharField(max_length=30, default="ebay_au")
    name = models.CharField(max_length=120)
    is_active = models.BooleanField(default=True)
    body_html = models.TextField(blank=True, default="")   # postage/returns/paym
    notes = models.TextField(blank=True, default="")

    class Meta:
        ordering = ["channel", "name"]
```

Edited in Django admin; read-only via the API. Wording changes never require a deploy.

---

3. Generators & boilerplate

3.1 `safe_context(item)` — the structural guard (D-52)

```
def safe_context(item) -> dict:
    """The ONLY data source for generators + the specifics builder.
    Built from an explicit ALLOWLIST (title, condition display, category/profile,
    cert_no, and the schema fields each profile declares safe). BLOCKED_FIELDS ar
    never copied in — even if set on the item — so templates physically cannot
    leak them. Tests assert: item with imei/serial/costs set → none appear in
    safe_context() output or any generated text."""
```

This is allowlist-first; `BLOCKED_FIELDS` **is the test oracle and the documentation of intent.**

3.2 Generator contract + registry

```
class ListingGenerator(Protocol):
    profile_key: str   # matches the Sprint 4 schema registry
    def title(self, ctx: dict) -> str: ...                        # target ≤ EBAY_
    def description_html(self, ctx: dict, *, specifics, boilerplate_html, sku_foo
```

```
GENERATORS = {"phones": ..., "stamps": ..., "coins": ..., "gold": ..., "generic":
def generator_for(item): ...   # profile lookup; ALWAYS falls back to "generic" (
```

Pure functions over `safe_context()` **— deterministic, no AI, no HTTP, no catalogue fetch.**

Title templates (skip blank parts; collapse whitespace; truncate to 80 on word boundary):

profile	template
phones	`{brand} {model} {storage_gb}GB {colour} {network_status} — {condition}`
stamps	`{country} {year} {denomination} {topic_theme} stamp {primary catalogue_ref}`
coins	`{country} {year} {denomination} {ruler_or_reign} coin {grade} {cert_no?}`
gold	`{weight_g}g {metal} {form} {fineness or karat display}`
generic	`item.title` → else `{category} — {sku}` (the fail-soft floor)

Description structure (D-50; all profiles, sections skipped when empty except phones' Faults which always renders):

1. `<h2>Overview</h2>` **— template sentence from attributes**
2. `<h2>Condition</h2>` **— item condition display + condition notes**
3. `<h2>Faults</h2>` **— buyer-material defects (phones: always present, "None noted" if empty)**
4. `<h2>What's included</h2>`
5. **Item-specifics** `<table>`
6. **Boilerplate** `body_html` **(postage/returns/payment)**
7. Optional SKU footer `<p>` **— only when** `include_sku_footer`

HTML rule (D-51): generator output uses **only** `ALLOWED_HTML_TAGS`. No `<a>` anywhere in generated output (eBay AU links policy). A unit test parses every generated description and asserts the tag set ⊆ whitelist. (Human edits to `description_html` are

Regan's own text — not sanitized in Sprint 5; the leak scan still runs over them.)

3.3 Regenerate semantics (D-50)

- `PATCH` changing `title` / `description_html` sets the matching `*_edited` flag.
- `POST …/generate/ {"fields": ["title","description","specifics","price"], "confirm_overwrite": false}`:
  - unedited/empty fields → regenerated freely (flags reset to `False`, `generated_meta` updated);
  - a requested field whose `*_edited` flag is set → **rejected with a validation error naming the protected fields** unless `confirm_overwrite: true`.
- `price` regeneration re-defaults from the current `ValuationReport.suggested_price` and rewrites `price_source`.

---

4. Item-specifics builder (`specifics.py`)

```
def build_specifics(item) -> list[dict]:   # [{ "name": str, "value": str }]
```

Per-profile mapping from `safe_context()` → buyer-facing specific names; blanks skipped; human edits rows afterwards in the UI (stored on the draft, never regenerated without the §3.3 flow):

- **phones:** Brand, Model, Storage Capacity, RAM, Colour, Network, Condition
- **stamps:** Country, Year of Issue, Denomination, Topic/Theme, Catalogue Number(s)
- **coins:** Country, Year, Denomination, Ruler/Reign, Grade, Certification Number, Composition, Weight
- **gold:** Metal, Form, Weight, Fineness/Karat
- **generic:** Condition (+ nothing invented)

Structurally cannot include `BLOCKED_FIELDS` (same `safe_context` source).

---

5. Readiness checker + leak scan (`readiness.py`)

```
@dataclass
```

```
class Check: key: str; level: str  # "pass" | "warn" | "fail"
            ; message: str
def check_readiness(draft) -> list[Check]:   # pure function
```

Fails (any one blocks `ready`):

key	condition
`no_photos`	no `photo_ids` selected
`title_missing` / `title_too_long`	blank / > `EBAY_TITLE_MAX` (80)
`description_missing`	blank `description_html`
`price_invalid`	`price` is null or ≤ 0
`quantity_invalid`	`quantity` ≤ 0
`item_unavailable`	item status is sold / archived
`leak_imei` / `leak_serial`	**leak scan hit** (below)
`photos_unresolvable`	any `photo_ids` entry no longer resolves to a usable processed photo (message lists them)

Warns (advisory only):

fewer than 3 photos · condition missing/unknown · category missing · `item_specifics` empty · no `price_source` recorded for the price · price below `min_acceptable_price` (when set) · price below true cost (when computable) · no boilerplate selected · phones: `faults` attribute absent/empty · description lacks the "What's included" heading.

Leak scan (D-52, Q-10 caveat 2): case-insensitive literal substring search of `title` + `subtitle` + `description_html` for the item's actual `imei` and `serial_no` values (only when those values exist and are ≥ 6 chars). **Fail level.** Cost/profit figures are **not** prose-scanned — their protection is structural (`safe_context`) and test-asserted; a test explicitly confirms `"$45 postage"` in a description does **not** trip the scan.

Gating: `PATCH status="ready"` is validated against zero fails (serializer). Export works from **any** status (D-53) — the UI badges an unready export. **Mark item listed** = the existing item `PATCH` (status → listed) behind a confirmation dialog in the UI; no new endpoint, never automatic.

6. Export (`export.py`)

`GET /api/listing-drafts/{id}/export/` → `application/zip` ; on success sets `exported_at = now()` and `status = "exported"` (from any status).

Bundle contents (D-54):

```
title.txt
subtitle.txt              # only if non-empty
description.html
specifics.csv             # header: name,value
readiness.json            # full Check list at export time + skipped_photo_ids
listing-summary.json      # all draft fields + item sku + photo manifest — the Ph
photos/01_<sku>.jpg …     # ordered, PROCESSED variants (not originals); missing
```

Clipboard blocks (copy title / copy description / copy specifics rows / copy boilerplate) are frontend-only.

---

7. Serializers & API endpoints

- `ListingDraftSerializer` — read: all fields + computed `readiness` summary (fail/warn counts); write: PATCH flips `*_edited` flags on title/description changes; `status="ready"` validated against zero fails.
- `ListingBoilerplateSerializer` — read-only.
- Create flow (`POST /api/items/{id}/listing-drafts/`): defaults `photo_ids` ← item photos in order, `item_specifics` ← builder, `title` / `description_html` ← generators (flags `False`), `price` ← current `ValuationReport.suggested_price` (+ `price_source` snapshot; null-safe when no valuation).

Method	Path	Purpose
GET / POST	`/api/items/{id}/listing-drafts/`	List the item's drafts / create with generated defaults.
GET/PATCH/DELETE	`/api/listing-drafts/{id}/`	Edit (edit-flag tracking; `ready` validated) / delete.
POST	`/api/listing-`	Regenerate fields; `confirm_overwrite` semantics

	`drafts/{id}/generate/`	**(§3.3).**
GET	`/api/listing-drafts/{id}/readiness/`	**Full Check list.**
GET	`/api/listing-drafts/{id}/export/`	**Zip bundle; sets** `exported_at` **+** `status=exported` **.**
GET	`/api/listing-boilerplates/` **(+** `/{id}/` **)**	**Read-only (admin-managed).**

---

8. Frontend — the Listing panel

New collapsible **Listing** section on Item Detail (cockpit pattern from Sprint 2). Components: `ListingPanel` (drafts list w/ status badges + create), `DraftEditor` (title input with live char counter that flags > 80, subtitle, **textarea + rendered-preview toggle — explicitly not a WYSIWYG**, format/price/currency/quantity/shipping-note, boilerplate select, SKU-footer toggle, per-field Regenerate buttons with a confirm dialog when the field is edited), `SpecificsEditor` (add/remove/edit rows), `PhotoPicker` (select + reorder from item photos), `ReadinessChecklist` (grouped fails/warns/passes + refresh), `ExportButtons` (the four copy blocks + Download zip, with an **unready badge** when fails remain), `MarkListedButton` (post-export; confirmation → item status PATCH). New API module: `listing.ts` .

---

9. Acceptance criteria

Done when:

1. New `listing` migrations **apply cleanly on real PostgreSQL in CI** (D-22 discipline); `manage.py backup` includes both new tables; restore on an empty DB passes.
2. Creating a draft from the cockpit produces a generated title (≤ 80 or flagged), structured description, specifics rows, ordered photos, and a price defaulted from the current valuation **with** `price_source` **recorded in** `generated_meta` **.**
3. Generated description HTML for every profile uses **only** `ALLOWED_HTML_TAGS` **—** verified by a parsing test; **no** `<a>` **ever appears in generated output.**
4. **Structural privacy guard proven:** an item with `imei` , `serial_no` , and all cost fields set produces generated title/description/specifics containing **none of them**;

`safe_context()` exposes no `BLOCKED_FIELDS` key. `cert_no` appears where the coin template uses it; SKU appears **only** when the footer toggle is on.

5. **Leak scan:** manually pasting the item's IMEI or serial into the description yields a readiness **fail**; `"$45 postage"` does not.
6. Regenerate fills unedited fields freely and **refuses to overwrite edited fields without `confirm_overwrite`.**
7. Readiness returns every fail/warn in §5 under test fixtures; `status="ready"` is rejected while any fail exists.
8. Export from any status downloads a zip matching §6 exactly (incl. `01_<sku>.jpg` naming, processed variants, skipped-ID reporting); `exported_at` + `status` are set.
9. Boilerplate is admin-editable, API read-only, and appended to generated descriptions.
10. **No HTTP / eBay / AI anywhere in `apps/listing`** (no network imports; grep-level check in review).
11. Backend + frontend suites green; **"Sprint 5 PostgreSQL Validation" CI gate green** before Sprint 5 (and Phase 3) is closed.

---

10. Test plan

Backend: model defaults/choices; `safe_context` allowlist (blocked keys absent even when set — the D-52 oracle test); per-profile titles incl. truncation + generic fail-soft; description section structure + whitelist parsing + no-anchor assertion; SKU footer toggle; specifics mapping + blank-skipping; leak scan positive (imei, serial) / negative (`"$45 postage"`, short values); every readiness fail + warn; ready-transition blocked with fails; regenerate edit-flag + `confirm_overwrite` semantics; price default + `price_source` snapshot (and null-valuation path); export zip contents/naming/skip-reporting + status/`exported_at`; boilerplate read-only API; draft CRUD; unauthenticated rejected; backup→restore includes new tables.

Frontend (light): char counter flags > 80; regenerate confirm dialog appears only when edited; `ReadinessChecklist` groups levels; `ExportButtons` shows the unready badge; `listing.ts` builds correct requests.

CI: "Sprint 5 PostgreSQL Validation" workflow (migrate + seed + suites on a Postgres service) — green before close.

---

11. Seed data (extend `seed.py`, idempotent)

- One `ListingBoilerplate` — `channel="ebay_au"`, name `"eBay AU — PLACEHOLDER boilerplate"`, generic postage/returns/payment wording, **clearly labelled placeholder** (comment: replace with Regan's real wording in admin; no links).
- One `ListingDraft` on the Samsung Galaxy S21 seed item, created through the real generator/specifics/price-default path, so the cockpit shows a working example.

---

12. Out of scope for Sprint 5 (restated — D-55)

eBay write/publish/OAuth/Business Policies/EPS uploads · eBay metadata or required-item-specifics lookups · AI/OCR generation · `SaleRecord` / realized-profit dashboards · auction scheduling / relisting automation · `SalesChannel` model (channel + `channel_data` is the seam) · bulk-lot composite listings · HTML sanitization of human-edited text. Each has a seam; filling it is a later phase.

Hand-off notes for the implementing agent

- **`safe_context()` is the security boundary.** Never let a template or the specifics builder read the item directly; never widen the allowlist without a spec change. If a template "needs" a blocked field, that is a design error — stop.
- The HTML whitelist is the entire generated-HTML story — no sanitizer library, no anchors, no exceptions.
- Regenerate must be boringly safe: when in doubt, refuse and ask for `confirm_overwrite`.
- Export is files only. If you find yourself reaching for an HTTP client anywhere in `apps/listing`, you have left the sprint.
- When ambiguous, prefer the smaller implementation that satisfies §9.